A MORL4WATER TOOLKIT

In this section, we present an overview of our environment setup, outline the major components, and demonstrate how the rivers discussed in the previous section can be modeled using our approach.

A.1 Water Management System

Our framework includes a `WaterManagementSystem` class that implements the MO-Gymnasium API [11]. To model specific components of the water systems, it employs additional classes organized into three categories: flows, facilities, and controlled facilities. Figure [add figure] illustrates the overall structure of the framework, highlighting the relationships among all its components.

To set up this class we need to inut list of water system classes defined, from up stream to downstream, the initial values of rewards we consider in the simulation, the starting date of the simulation, time-step size - how ofter the decisions are made and optionally whether we want to include timestamp in the observation.

Nile. In the Nile simulation, the starting date is January 1st, 2025, and decisions are made monthly over a span of 20 years. This results in a total of 240 timesteps per episode. However, the number of timesteps may be fewer if the episode terminates early, such as when the water level in a facility drops below 0 or exceeds its maximum capacity.

Sesquehanna. In the Susquehanna simulation, decisions are made every 4 hours over the course of a year, starting on January 1st, 2021. This start date was selected to align with the original model [46], where the first day of the year, a Friday, played a critical role due to release decisions being influenced by the day of the week. The total number of steps per episode is 2190, but the episode may terminate earlier based on the water levels in facilities.

A.2 Actions

The actions represent water releases from each dam modeled in the system, making the dimension of the action space dependent on the number of reservoirs modeled in the Water Management System (WMS). Additionally, each reservoir can have multiple outflows. The WMS requires normalized actions, so the user must specify the maximum release as part of the release constraints. Consequently, the action space is defined as $\text{Box}[0, 1]$.

Nile. The actions for the Nile simulation consist of four dimensions, with each dimension representing the water release from one of the following dams: GERD, Roseires, Sennar, and HAD.

Susquehanna. Release decisions at Conowingo are divided into four directions: Baltimore, Chester, the atomic plant, and downstream (for hydroelectricity and further flow of the Susquehanna). Therefore, the actions for the Susquehanna simulation also consist of four dimensions, each corresponding to a different destination.

A.3 Observations

Observations in the Water Management System (WMS) represent the amount of water in each reservoir (controlled facility), so the dimensionality depends on the number of controlled facilities in the system. By default, the observation is represented as water volume in each reservoir is represented in liters; however, it can also be expressed as water level (e.g., in meters) by modifying the step function. This information is accessible in the info dictionary. Since observation values can be large, they are normalized to reflect the percentage of reservoir capacity, requiring the maximum storage volume during the setup of this class. Thus, the observation space is also defined as $[0, 1]$. Additionally, a timestamp can be included to represent the month, hour, or day of the year, which increases the dimensionality of the observation space by one; these timestamps are also normalized to the range $[0, 1]$.

Nile. The Nile is modeled using the metric system, with observations representing the water volume in four reservoirs, measured in cubic meters. The maximum storage volumes used to normalize the observation space are provided in the Table 7. To capture seasonality, the month of the year is also included as an additional observation.

Susquehanna. The Susquehanna simulation is modeled using the imperial system. Unlike Nile simulations, the original model [14] represents the state of the system by water levels in the reservoir rather than its volume. Consequently, the step method was modified to account for water level data. In this simulation, the observation includes both the water level in the Conowingo reservoir and, similar to the Nile model, a timestamp. However, only the hour of the day is included, as some release rules for the Susquehanna are time-dependent. The maximum water levels used for normalizing the observations are provided in the Table 8.

A.4 Rewards (Objectives)

As a multi-objective problem, the simulation returns a vector of rewards, with the vector's dimension determined by the number of objectives defined. To compute rewards, our framework implements various objective functions, reflecting the diversity of objectives commonly found in the literature for different systems. Rewards can be defined for each component of the Water Management System (WMS). For instance, if the goal is to maximize hydropower production, the objective can be specified in the `PowerPlant` class. The framework offers different

reward functions depending on the goal: in some cases, we may want to maximize production value, while in others, ensuring a minimum level of power generation is the priority. Rewards can also be normalized by providing the maximum value during facility setup.

In practice, objectives are managed through two key components: rewards and objective functions. The `determine_reward` method, implemented for each `Facility` and `ControlledFacility` class, calculates rewards by determining which class attributes (e.g., water demand, inflow, or reservoir water levels) are relevant for the objective. It then applies the `objective_function` attribute, passed during class initialization, to compute the final reward from the selected components. The `Objective` class offers nine predefined objective functions to handle various calculations, providing flexibility across different scenarios (the list of of all reward functions can be found in the Table 5).

The simulation's objectives can be easily modified by overriding the `determine_reward` method to use a different set of attributes or by swapping the `objective_function` during initialization. Furthermore, objectives across different classes can be combined by setting the same `objective_name` attribute, allowing multiple components to contribute to a shared goal.

Reward function	**Description**
`no_objective()`	Returns a constant value of 0.0, used as a placeholder for no objective.
`identity(value)`	Returns the *input value* as-is, representing a simple identity function.
`is_greater_than_ minimum(minimum_ value)`	Returns 1.0 if the *input value* is greater than or equal to the specified minimum, otherwise returns 0.0.
`is_greater_than_ minimum_with_ condition(minimum_ value)`	Similar to `is_greater_than_minimum`, but also checks an additional condition. Returns 1.0 if the condition is true and the value meets the minimum requirement.
`deficit_minimised (demand, received)`	Minimizes the deficit by returning the negative difference between demand and received values if the demand is unmet, otherwise returns 0.0.
`deficit_squared_ ratio_minimised (demand, received)`	Minimizes the square of the ratio of deficit to demand, returning a value that penalizes larger deficits.
`scalar_identity (scalar)`	Returns the *input value* multiplied by a constant scalar.
`sequential_scalar (scalar)`	Multiplies the *input value* by a scalar from a list based on a sequential index.
`supply_ratio_ maximised(demand, received)`	Maximizes the ratio of received to demand, capped at 1.0 if the received value meets or exceeds demand.

Table 5: Reward Functions and Descriptions. Note that *input value* is inflow/outflow of a facility.

A.4.1 Nile. For the Nile Water Management System, we consider four objectives, as outlined in the main paper. Here, we present the mathematical definitions of these objective functions, based on the formulation from [34]. It is important to note that in our implementation of the Nile WMS, all objectives are normalized by defining the maximum achievable value for each objective at every timestep.

Egypt irrigation deficit is calculated as the monthly value of unmet demand in Egypt:

$$Irr_{Egypt} = -\max(0, D^t_{\text{Egypt}} - V^t_{\text{Egypt}}) \tag{4}$$

where D^t_{Egypt} is the water demand in month t and V^t_{Egypt} is the received water flow by the irrigation district Egypt in month t.

Sudan irrigation deficit is calculated as monthly value of unmet demand in Sudan:

$$Irr_{Sudan} - \max(0, D^t_{\text{Sudan}} - V^t_{\text{Sudan}}) \tag{5}$$

where D^t_{Sudan} is the water demand in month t and V^t_{Sudan} is the received water flow by the irrigation district Egypt in month t.

Egypt min HAD level indicates the water reliability in the HAD, determining whether the water level exceeds the minimum required for power generation:

$$\text{HAD}_t = \begin{cases} 1, & \text{if } h_{\text{HAD}}(s_t) \geq 159 \\ 0, & \text{otherwise} \end{cases} \tag{6}$$

where $h_{\text{HAD}}(s_t)$ is the function that calculates the water level in HAD in meters above sea level (masl) in the month t when the storage is s.

Ethiopia hydropower is Ethiopia's monthly hydroenergy generation from GERD power plant:

$$HP_{Ehiopia} = P^t_{\text{GERD}} \cdot hrs_t \tag{7}$$

, where hrs_t is the number of hours in the month t and P^t_{GERD} is the power output per hour calculated as follows:

$$P^t_{\text{GERD}} = \min\left(r^t_{\text{GERD}}, r^{\max}_{\text{GERD}}\right) \cdot g \cdot \max\left(0, h_{\text{GERD}}(s^t) - h^{\text{turbine}}_{\text{GERD}}\right) \cdot \eta_{\text{GERD}} \tag{8}$$

where $r^{\max}_{\text{GERD}}$ is the flow that corresponds to the maximum rotational speed of the turbine, g is the gravitational constant, $h^{\text{turbine}}_{\text{GERD}}$ is the height of the turbines in GERD and η_{GERD} is the eficiency of the hydropower generation process in GERD.

A.4.2 Susquehanna. For the Susquehanna Water Management System, we consider six objectives, as outlined in the main paper. Here, we present the mathematical definitions of these objective functions, based on the formulation from [14], [32] and [46]. It is important to note that in our implementation of the Susquehanna WMS, all objectives are normalized by defining the maximum achievable value for each objective at every timestep.

Recreation is the storage reliability throughout the tourist season's weekends (June-August), which is calculated as the number of weekend days in the peak season that are equal or larger than the intended level to ensure recreational activities - 32.5 m (106.5 ft):

$$R_t = \begin{cases} 1, & \text{if } is_summer_weekend(t) \wedge h_{\text{Conowingo}}(s_t) \geq 32.5, \\ 0, & \text{otherwise} \end{cases} \tag{9}$$

where $is_summer_weekend(t)$ is a function, which checks if the current date t is a summer weekend and $h_{\text{Conowingo}}(s)$ is the function that calculates the water level in Conowingo in meters above sea level (masl) when the storage is s_t.

Environmental shortage shortage index in relation to the minimum flow required by FERC The quadratic formulation discourages large water deficits while allowing for minor shortages [17]

$$Env = \frac{\max(Z_t - Y_t, 0)^2}{Z_t} \tag{10}$$

where Y_t (m^3) is the water release at time t and Z_t (m^3) is the corresponding FERC flow requirement.

Water supply reliability to the Atomic Power Plant, Chester and Baltimore, which is volumetric reliability defined as:

$$Rel_{it} = \frac{Y_{it}}{D_{it}} \tag{11}$$

where Y_{it} (m^3) is the water delivery at time t, D_{it} (m^3) is the corresponding demand, and i represents the water supply to Baltimore, Chester, or the Atomic Power Plant.

Hydropower revenue is defined as the economic revenue gained from hydropowerproduction at the Conowingo hydropower dam in US$/MWh:

$$HPR_{Conowingo} = P^t_{\text{Conowingo}} \cdot t_{sim_step} \cdot \rho_t \tag{12}$$

where $P^t_{\text{Conowingo}}$ is power generated calculated similarly as in the equation 8, ρ_t is energy price defined by the seven-hour moving average of the energy price trajectory in the Pennsylvania, New Jersey–Maryland energy market [39], and t_{sim_step} is the number of hours in the simulation timestep (4 hours).

A.5 Flow

The `Flow` class represents water flow within the system, enabling users to define connections between different facilities. It supports multiple inflow sources and outflow destinations, allowing for the creation of complex water flow networks. The class is easily customizable through a few parameters, making it adaptable to the specific needs of different simulations:

- `max_capacity`: The maximum capacity of the flow. Can be used to model flooding scenarios by providing negative rewards or terminating the simulation. Additionally, it can be set to infinity to disable the feature.
- `evaporation_rate`: The evaporation rate of the flow that causes water loss.
- `delay`: The number of time steps it takes the water to get from the source to the destination, it can be used to model long connections but is optional
- `default_outflow`: The default amount of water flowing through the flow when the `delay` parameter is used. The `default_outflow` is used for the first number of steps equal to the value set for the `delay` parameter.

Nile. The inflow to the GERD reservoir is defined using the Inflow class to capture water entering from the Blue Nile using data generated stochastically, based on Sari [34] (detailed data description can be found in the Table 7). This inflow is modeled with unlimited capacity to represent natural inflow conditions. Water is then released from GERD to the associated power plant through a defined flow, which supports hydropower generation.

Downstream flows are established between key facilities, such as the flow from GERD to the Roseires reservoir, which combines releases from GERD and additional catchment inflows. Similarly, flows connect other reservoirs, such as Sennar and HAD, to irrigation systems and power plants, allowing for the simulation of water distribution across multiple sectors.

Several catchments are modeled to contribute local inflows to the system. These catchments, represented by specific inflow data, ensure that the model reflects the geographical and environmental variability of water supply along the Nile.

In addition to the inflows and outflows of major reservoirs, the model incorporates flows to various irrigation districts, such as Gezira and DSSennar, which are essential for agricultural water management in Sudan and Egypt.

Susquehanna. The primary reservoir in the system, Conowingo, receives inflows from two main sources: the main channel of the Susquehanna River and lateral inflows. These inflows are modeled using the Inflow class, which captures water entering the reservoir from different sources. The inflow data used comes from [46] and detailed description can be found in the Table 8.

Water is released from Conowingo to various destinations, including the Atomic plant, Baltimore, Chester, and the power plant. The releases are controlled through the Flow class, which defines how water is distributed among these destinations. Additionally, the power plant releases water downstream, contributing to further flow management.

A.6 Facility

Both the `Facility` and `ControlledFacility` classes represent components that influence the water system and generate rewards based on their state. While `Facility` models static elements that passively respond to the environment, `ControlledFacility` allows the agent to take actions that actively affect the system. Our framework currently includes three facility types — `Catchment`, `DemandDistrict`, and `PowerPlant`—and two controlled facility, the `Reservoir` and `ReservoirWithPump`.

A.7 Catchements

The `Catchment` class is a basic utility for modeling additional inflows into the system. It can be initialized using the `all_water_accumulated` parameter, which accepts a list of inflow values provided by the catchment at each simulation step.

Nile. The simulation includes several catchments to model additional water inflows at key points along the Nile. The GerdToRoseires Catchment adds local inflows between GERD and the Roseires Reservoir. The RoseiresToAbuNaama Catchment introduces inflows downstream of Roseires to support irrigation. The SukiToSennar Catchment contributes water between the Upper Sennar and Sennar Reservoir. The Dinder and Rahad Catchments provide tributary inflows to the Gezira Irrigation System. The White Nile Catchment represents the inflow from the White Nile, joining the Blue Nile at Khartoum. Finally, the Atbara Catchment models the inflow from the Atbara River before reaching the High Aswan Dam in Egypt.

Susquehanna. In our Susquehanna model, catchments are not included.

A.8 Demand District

The `DemandDistrict` class represents a facility that consumes water to meet its requirements. Its default implementation calculates a reward based on the proportion of demand that is fulfilled. The water demand for each simulation step is specified through the `all_demand` initialization parameter.

Nile. In the Nile River simulation, demand districts are modeled to represent areas that consume water to meet agricultural and consumptive demands. Each district, such as DSSennar, Gezira, Hassanab, Tamaniat, USSennar, and Egypt, has a predefined water demand for each simulation step, which is specified through input data files. The reward for each district is calculated based on how well the water demand is met, with a focus on minimizing deficits. This is achieved through the deficit_minimised objective, which penalizes unmet water needs.

Susquehanna. In the Susquehanna River simulation, demand districts are represented as regions such as Baltimore, Chester, and Atomic, each with predefined water demands. These demands are specified for each simulation step, and the objective is to maximize the ratio of water supplied to demand.

A.9 Power Plant

The `PowerPlant` class models a facility capable of generating electricity from water flow. It requires a connection to a reservoir that supplies the stored water necessary for power generation. Additionally, the class includes multiple parameters to define the physical characteristics that are critical for calculating its power production capacity.

- `efficiency`: Efficiency coefficient of the power plant.
- `head_start_level`: The water level required for the power plant to generate water.

- `max_capacity`: Maximum power capacity of the power plant.
- `operating_hours`: The number of hours the power plant operates for.
- `min_turbine_flow`: Minimum flow through the turbines to produce hydro energy.
- `max_turbine_flow`: Maximum flow through the turbines for hydro energy production.
- `water_usage`: Ratio of water flowing through the turbines that is consumed.

Nile. In the Nile simulation, the following power plants are modeled, each with distinct characteristics and roles within their respective regions:

- **GERD Power Plant (Ethiopia)**: The GERD (Grand Ethiopian Renaissance Dam) Power Plant is connected to the GERD Reservoir in Ethiopia. It operates with an efficiency of 93% and a maximum turbine flow of 4320 cubic meters per second. The power plant has a maximum capacity of 6000 MW. Water from the GERD reservoir is released through the turbines to generate electricity. The energy output is normalized based on the maximum potential monthly production, ensuring a realistic evaluation of its energy generation capabilities.
- **Roseires Power Plant (Sudan)**: This plant is linked to the Roseires Reservoir in Sudan. Although the specific parameters of the power plant are not detailed in the code, it receives water from upstream facilities like the GERD Power Plant and contributes to Sudan's energy supply.
- **Sennar Power Plant (Sudan)**: The Sennar Power Plant is connected to the Sennar Reservoir in Sudan, which has a maximum capacity of 579.9 million cubic meters.
- **High Aswan Dam (HAD) Power Plant (Egypt)**: The HAD Power Plant is linked to the High Aswan Dam Reservoir (HAD) in Egypt, which has a storage capacity of 182.7 billion cubic meters. The plant's operation focuses on electricity generation and maintaining a minimum water level of 159 meters.

Susquehanna. In the Susquehanna River simulation, the Conowingo Power Plant is linked to the Conowingo Reservoir Muddy Run Pump and operates to maximize energy revenue, using 13 turbines with a maximum flow rate of 85,412 cfs and 79% efficiency.

A.10 Reservoir

The `Reservoir` class manages water flow within the system, allowing for the accumulation and controlled release of water. It supports the modeling of complex storage geometries, such as conical shapes, using functions that define the relationship between stored water volume, water level, and surface area. These functions are specified through sets of data points, which are subsequently interpolated by the `Reservoir` class:

- `storage_to_minmax_release_relation`: Relation of the stored water amount in m^3 to the minimum and maximum amount of water that can or needs to be released from the reservoir.
- `storage_to_level_relation`: Relation of the stored water amount in m^3 to the water height level in meters, used for calculating power production.
- `storage_to_surface_relation`: Relation of the stored water amount in m^3 to the surface area in m^2, used for calculating the amount of evaporated water.

Additionally, the `Reservoir` class can be configured with several key parameters that define its attributes:

- `max_capacity`: The maximum volume of water the reservoir can hold.
- `stored_water`: The initial water volume stored in the reservoir.
- `evap_rates` and `evap_rates_timestep_size`: Evaporation rates per simulation step, representing environmental effects on water storage with indication of size of the timestep (e.g., daily, monthly) of the time series
- `integration_timestep_size`: The time interval for updating the reservoir's state.

Finally, the reservoir can manage water release in multiple directions by receiving an action for each connected destination facility. It calculates the total possible outflow and determines the proportion of water released to each destination (`split_release_ratio`) based on the ratio of the received actions. This ratio is then used by the `Flow` class to distribute the water to downstream facilities accordingly.

Some reservoirs include built-in pumps that can store water and pump it in or out to support additional functions, such as hydropower generation or recreational activities. To model these reservoirs, we define the `ReservoirWithPump` class. In addition to the parameters required for a standard reservoir, this class requires pump-specific data, such as the pump's maximum and initial capacity, its evaporation rates, and the relationships between its storage, level, and surface area. It also requires a pumping rule function, which determines the amount of water pumped or released based on predefined rules. Note that the pump's releases are not optimized.

Nile. In the Nile simulation, four reservoirs are defined to manage water flow, each operating with monthly evaporation rates and a 4-hour integration timestep. For detailed descriptions of the data used to model these reservoirs, refer to the Table 7.

Susquehanna. In the Susquehanna simulation, Conowingo is modeled as a reservoir with a pump (Muddy Run), where pumping rules depend on the time of day and day of the week. With release decisions made every 4 hours, the integration timestep is set to 1 hour. Detailed data and the pump rules are provided in the Table 8.

A.11 Data Required

To accurately model the water systems in our WMS framework, specific meteorological and physical data is required for each component. Table 6 provides an overview of data required to set up a WMS, its type and location in the model.

Data	Class	Type	Description
max capacity	Reservoir	Scalar	Maximum reservoir capacity (m^3)
max release	Reservoir	Scalar	Maximum release rate (m^3/s)
initial storage	Reservoir	Scalar	Initial water storage (m^3)
evaporation rates	Reservoir	Time Series	Evaporation rate per step as a percentage of surface area
storage to min/max release	Reservoir	Tabular Data	Mapping from stored water (m^3) to release bounds
storage to level	Reservoir	Tabular Data	Water level (m) as function of stored volume
storage to surface	Reservoir	Tabular Data	Surface area (m^2) vs stored volume, for evaporation
initial storage (pump)	Reservoir	Scalar	Initial pump storage (m^3)
evaporation rates (pump)	Reservoir	Time Series	Pump evaporation per step (as surface % loss)
storage to level (pump)	Reservoir	Tabular Data	Pump water level (m) vs stored volume
storage to surface (pump)	Reservoir	Tabular Data	Pump surface area (m^2) vs stored volume
pumping rules	Reservoir	Function	Function governing release and pumping based on time, storage, etc.
inflows to pump	Reservoir	Time Series	Inflow rate to pump (m^3/s)
efficiency	PowerPlant	Scalar	Efficiency of power generation
min turbine flow	PowerPlant	Scalar	Minimum flow for turbine operation
max turbine flow	PowerPlant	Scalar	Maximum turbine throughput
head start level	PowerPlant	Scalar	Required head level for power generation
max capacity	PowerPlant	Scalar	Maximum output capacity
n_turbines	PowerPlant	Scalar	Number of turbines
turbines	PowerPlant	Tabular Data	Min/max flow rates for each turbine (if $n_turbines > 1$)
water demand	IrrigationDistrict	Time Series	Water demand per simulation step
inflow	Inflows, Catchment	Time Series	River inflow rate per step

Table 6: Data required for configuring a water management system, including associated classes, data types, and descriptions.

Nile. Table 7 showcases the description of the data used to set up the environment for Nile WMS with sources.

Data	Facility	Value	Source
inflow	Catchments in Sudan	Monthly time series located in examples/-data/nile	Stochastic data generated by [40]
efficiency	GERD Power Plant	93%	[1]
head start level	GERD Power Plant	507 m	[8]
max capacity	GERD Power Plant	6000 MW	[40]
max turbine flow	GERD Power Plant	4320 m	[1]
min turbine flow	GERD Power Plant	0 m	[34]
n_turbines	GERD Power Plant	1	[40]
initial storage	GERD reservoir	15BCM	[40]
max capacity	GERD reservoir	117.5BCM	[40]
max release	GERD reservoir	10000 m^3/s	[34]

Table 7: Summary of data, facilities, values, and sources for the water management simulation system.

Data	Facility	Value	Source
evaporation rate	GERD, Sennar, Roseires and HAD reservoirs	Monthly time series located in examples/-data/nile	[40]
efficiency	HAD Power Plant	90%	[34]
head start level	HAD Power Plant	147 m	[40]
max capacity	HAD Power Plant	2100 MW	[40]
min turbine flow	HAD Power Plant	0 m	[40]
n_turbines	HAD Power Plant	1	[40]
initial storage	HAD reservoir	137BCM	[34]
max capacity	HAD reservoir	182.7BCM	[40]
max release	HAD reservoir	7000 m^3/s	[34]
max turbine flow	HAD Power Plant	4211 m	[40]
water demand	Irrigation districts	Monthly time series located in examples/-data/nile	[34], based on population growth and historical data
initial storage	Roseires reservoir	4.57BCM	[34]
max capacity	Roseires reservoir	6.1BCM	[40]
max release	Roseires reservoir	15000 m^3/s	[34]
initial storage	Sennar reservoir	0.43BCM	[34]
max capacity	Sennar reservoir	0.6BCM	[40]
max release	Sennar reservoir	7000 m^3/s	[34]
storage to level to surface relationship	Sennar, GERD, Roseires, HAD reservoirs	Tabular data	[40]
storage to min max release	Sennar, GERD, Roseires, HAD reservoirs	Tabular data	[40]
inflow	White Nile, Atbara, Blue Nile	Monthly time series located in examples/-data/nile	Data generated by [34] based on historical data gathered by [8]

Table 7: Summary of data, facilities, values, and sources for the water management simulation system.

Susquehanna. Table 8 showcases the description of the data used to set up the environment for Susquehanna WMS with sources.

Data	Facility	Value	Source
max release	Conowingo (with 4 outflows)	41.302169, 464.16667, 54.748458, 85412	[31]
initial storage	Conowingo reservoir with Muddy Run Pump	2.6BCM	[31]
max capacity	Conowingo reservoir with Muddy Run Pump	7BCM	[31]
evaporation rate	Conowingo reservoir with Muddy Run Pump	Daily evaporation rates for Conowingo found in `examples/data/susquehanna`	[31]
storage to min max release	Conowingo reservoir with Muddy Run Pump	Tabular data for Conowingo located in `examples/data/susquehanna`	[31]
storage to level to surface relationship	Conowingo reservoir with Muddy Run Pump	Tabular data for Conowingo in `examples/data/susquehanna`	[31]
initial storage of the pump	Conowingo reservoir with Muddy Run Pump	1.9BCM	[31]
evaporation rates of the pump	Conowingo reservoir with Muddy Run Pump	Daily evaporation rates for Muddy Run in `examples/data/susquehanna`	[31]
storage to level to surface relationship of the pump	Conowingo reservoir with Muddy Run Pump	Tabular data for Muddy Run Pump in `examples/data/susquehanna`	[31]
pumping rules	Conowingo reservoir with Muddy Run Pump	Implemented as function `muddyrun_pumpturb` in the main Susquehanna script	[31]
inflows to the pump	Conowingo reservoir with Muddy Run Pump	Daily inflow data to Muddy Run in `examples/data/susquehanna`	[31]
efficiency	Conowingo Power Plant	79%	[31]
head start level	Conowingo Power Plant	0	[31]
n_turbines	Conowingo Power Plant	13	[31]

Table 8: Data, facilities, values, and sources for the Conowingo water management system and associated components located in `examples/data/susquehanna/`.

Data	Facility	Value	Source
water demand	Atomice, Baltimore and Chester Irrigation Districts	Daily water demand in `examples/data/susquehanna`	[31]
turbines	Conowingo Power Plant	Tabular data with turbine min/max flows in `examples/data/susquehanna`	[31]
inflow	Later and Main Conowingo Inflow	Daily inflow time series in `examples/data/susquehanna`	[31]

Table 8: Data, facilities, values, and sources for the Conowingo water management system and associated components located in `examples/data/susquehanna/`.

B HYPERPARAMETER SET UP AND FULL RESULTS

Tables 9 and 10 provide an overview of the parameter configurations used for each algorithm and simulation across the experiments presented in this paper. Hyperparameters were tuned by randomly sampling configurations and evaluating their performance using hypervolume and expected utility metrics over training time. For EMODPS, we employed the NSGA-2 evolutionary algorithm with radial basis functions (RBFs), where the number of RBFs to construct is specified via the `num_rbf` parameter [13]. The algorithmic design of PCN differs significantly from that of GPI-LS and CapQL: while the latter are actor-critic methods, PCN is based on reward-conditioned policies. Consequently, `arch_net` denotes the architecture of the actor-critic networks, whereas `hidden_dim` refers to the dimensionality of the reward-conditioned policy network in PCN, which also requires a scaling factor for each reward and timestep. The `alpha` parameter, specific to CapQL, is an entropy regularization coefficient.

WMS	Algorithm	ϵ	n_rbf	γ
Nile	EMODPS	[0.5, 0.5, 0.5, 0.5]	9	1
Susquehanna	EMODPS	[0.5, 0.5, 0.5, 0.5, 0.5, 0.5]	6	1

Table 9: Hyperparameters for EMODPS in the Nile and Susquehanna water management systems (WMS).

WMS	Alg.	γ	learning _rate	batch _size	buffer _size	arch _net	hidden _dim	scaling_ factor	alpha	num_q _nets
Nile	PCN	1	0.0003	256	-	-	128	[0.5, 0.5, 0.5, 0.5, 0.5]	-	-
	GPI-LS	1	0.0003	128	400000	[32, 32]	-	-	-	2
	CapQL	1	0.0003	128	1000000	[128, 128]	-	-	0.02	2
Susquehanna	PCN	1	0.003	256	-	-	64	[0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5]	-	-
	GPI-LS	1	0.0003	128	400000	[64, 64]	-	-	-	2
	CapQL	1	0.0003	128	1000000	[256, 256]	-	-	0.2	3

Table 10: Hyperparameters for PCN, GPI-LS, and CapQL in the Nile and Susquehanna water management systems (WMS).

Given this parameter set up, we present results, together with standard deviation, in Table 11.

C SEED ANALYSIS FOR SUSQUEHANNA

To investigate whether MORL algorithms exhibit similar trends in exploring different regions of the Pareto front, as previously observed in the Nile environment, we conducted a comparable seed-based analysis. This analysis aims to further examine the distribution of solutions across different random seeds and to compare the exploration behavior of MORL algorithms with that of EMODPS. Figure 9 displays the solution sets for each algorithm across first two seeds (including all seeds would result in excessive visual clutter). Consistent with the Nile results, EMODPS demonstrates high consistency, with nearly complete overlap between solution sets from both seeds. In contrast, GPI-LS shows notable variation: Seed 2 produces solutions concentrated around the extreme values of the first five objectives with minimal trade-offs, while Seed 4 reveals more balanced trade-offs across objectives. Interestingly, CapQL exhibits similar coverage across the first two seeds, indicating more stable behavior. PCN, as observed in the Nile case, generates diverse regions of the Pareto front across different seeds, reflecting inconsistent exploration.

To quantitatively assess differences between solution sets generated by different seeds, we compute the pairwise average minimal distance between solution sets. Specifically, for each pair of seeds, we calculate the average Euclidean distance from each solution in one set to its nearest neighbor in the other. To summarize overall seed-wise variability, we report the mean of these distances as a single scalar value per algorithm (Table 12). Lower mean distances indicate greater consistency across seeds. EMODPS achieves the lowest mean distance (0.1094), reflecting highly stable behavior. GPI-LS and CapQL show

	Algorithm	Hypervolume (% Baseline) ↑	Expected Utility ↑	Cardinality ↑	Sparsity ↓
Nile	EMODPS (Baseline)	2.32E+08 (0.06E+08) (100%)	61.56 (1.98)	847 (195)	0.54 (0.39)
	PCN	6.49E+07 (8.63E+07) (28%)	28.24 (19.73)	49 (17)	58.26 (113.05)
	GPI-LS	2.08E+08 (0.07E+08) (90%)	58.66 (1.44)	28 (12)	508.16 (366.63)
	CapQL	4.89E+07 (6.13E+07) (20%)	28.56 (22.47)	32 (7)	196.94 (307.87)
Susquehn.	EMODPS (Baseline)	5.33E+17 (3.70E+17) (100%)	1012.15 (40.75)	969 (165)	51.83 (15.18)
	PCN	1.25E+15 (2.80E+15) (0.2%)	624.18 (77.26)	57 (10)	8139.42 (17818.57)
	GPI-LS	9.74E+16 (8.61E+16) (18%)	609.88 (290.46)	90 (7)	5532.58 (5857.00)
	CapQL	1.47E+16 (3.29E+16) (3%)	989.29 (20.96)	68 (7)	1870.71 (439.63)

Table 11: Comparison of MORL algorithms and EMODPS.

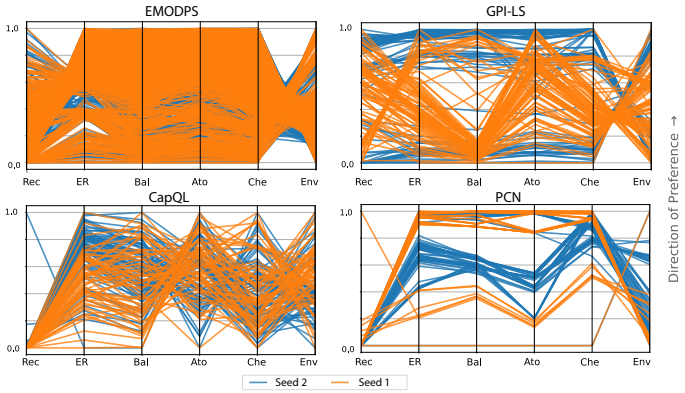

Figure 9: Solution sets for two selected seeds per algorithm.

similar values, while GPI exhibits a higher mean distance (approximately 0.30) compared to its result in Nile. PCN yields the highest value (0.53), consistent with the variability observed in the parallel coordinate plots.

Algorithm	Mean Distance
CapQL	0.3327 *(± 0.06)*
PCN	0.5317 *(± 0.20)*
GPI-LS	0.3206 *(± 0.12)*
EMODPS	0.1094 *(± 0.03)*

Table 12: Mean distance between solution sets for different algorithms for Susquehanna.